

Graph-Based Context Retrieval for Retrieval-Augmented Generation:

A Link-Prediction Study on the Wiki-Vote Network

Net RAG Project

Abstract

Retrieval-augmented generation (RAG) systems improve language-model reliability by grounding generation in external evidence, but conventional retrieval can miss structural context when relevance is expressed through indirect relationships rather than lexical similarity. This paper presents a compact GraphRAG-style retrieval module that formulates context retrieval as link prediction on the Wikipedia Vote Network. The system parses raw SNAP Wiki-Vote edges, performs a source-node-stratified random train/test split, samples negative candidate targets, and compares four graph scoring functions: Common Neighbors, Jaccard co-efficient, Adamic-Adar, and approximate Personalized PageRank. On the full dataset split with 82,822 training edges and 20,867 hidden test edges, Personalized PageRank obtains the best overall result, with Precision@10 of 0.239297, HitRate@10 of 0.791845, and MRR of 0.551049. The study shows that graph-based retrieval can provide a reproducible, inspectable retrieval layer for GraphRAG pipelines, even before adding a downstream language generator.

1 Introduction

Large language models can generate fluent answers while still failing when the required knowledge is absent, private, or stale. Retrieval-augmented generation addresses this limitation by retrieving external evidence before generation, allowing model outputs to be grounded in non-parametric memory [8]. However, a retrieval component based only on direct lexical or vector similarity can miss relevant context that is expressed through relationships. Recent GraphRAG work argues that graph indexes can improve retrieval and summarization when questions require global or relational context [2].

This project studies the retrieval part of that idea in isolation. Instead of constructing a full question-answering application, it builds a graph retrieval module over the Wikipedia Vote Network. A directed edge from user u to user v denotes that u voted for v in a Wikipedia administrator election. If an unobserved edge from u to v can be predicted from the training graph, then v is treated as structurally relevant context for u .

The contribution is practical and reproducible. The repository implements raw data parsing, deterministic edge splitting, candidate generation, four retrieval algo-

rithms, ranking evaluation, single-user context retrieval, and visualization. This paper summarizes the implemented method and reports the saved full-dataset results using the project artifacts.

2 Related Work

RAG combines a parametric generator with an external retrieval mechanism so that generation can be conditioned on retrieved evidence [8]. GraphRAG extends this direction by using graph structure as an indexing and aggregation substrate, enabling retrieval over relationships, communities, and indirect links [2].

The retrieval task in this project is closely related to classical link prediction. Liben-Nowell and Kleinberg formalized link prediction as inferring future or missing interactions from a network snapshot and showed that graph proximity measures can be predictive [9]. Common Neighbors and Jaccard-style overlap scores are simple local proximity measures, while Adamic-Adar weights rare common neighbors more strongly [1]. PageRank and Personalized PageRank provide global random-walk views of graph importance and source-specific relevance [4, 10].

The experimental dataset comes from SNAP’s Wikipedia Vote Network, whose source material is associated with work on signed social networks and link prediction in online communities [5–7]. The implementation uses NetworkX for graph construction and traversal [3].

3 Methodology

3.1 Problem Formulation

Let $G = (V, E)$ be the directed Wiki-Vote graph. The project hides a subset of edges E_{test} and builds a training graph $G_{\text{train}} = (V, E_{\text{train}})$. For a source node s , the retrieval module ranks candidate targets t that are not already visible as training outgoing edges from s . Hidden test edges are positive candidates, and randomly sampled non-existent edges are negative candidates.

The output is a ranked list:

$$R_s = \text{rank}_{t \in C_s} f(s, t),$$

where C_s is the candidate set for source s and f is a graph scoring function.

3.2 Data Preparation

The raw file `Wiki-Vote.txt` is parsed directly. Comment lines are skipped, edge rows are converted into integer node pairs, and duplicate edges are removed. Because the dataset does not include timestamps for each edge in the local task file, the project uses a deterministic random split. For each source node with at least two outgoing edges, approximately 20% of its targets are sampled into the test set, while at least one outgoing edge is kept for training. Sources with one outgoing edge remain in the training graph.

SNAP reports 7,115 nodes and 103,689 edges for the Wiki-Vote network [5]. The repository’s default split with seed 42 produces 82,822 training edges and 20,867 testing edges from 103,689 unique edges.

3.3 Graph Scoring Functions

The project evaluates four scoring functions. Common Neighbors counts neighbor overlap in an undirected projection:

$$f_{\text{CN}}(s, t) = |\Gamma(s) \cap \Gamma(t)|.$$

The Jaccard coefficient normalizes overlap by the union size:

$$f_{\text{Jaccard}}(s, t) = \frac{|\Gamma(s) \cap \Gamma(t)|}{|\Gamma(s) \cup \Gamma(t)|}.$$

Adamic-Adar discounts high-degree common neighbors:

$$f_{\text{AA}}(s, t) = \sum_{z \in \Gamma(s) \cap \Gamma(t)} \frac{1}{\log |\Gamma(z)|}.$$

These three scorers use the undirected training projection to reduce sparsity.

Personalized PageRank is implemented as an approximate random-walk scorer on the directed training graph. For each source node, the implementation simulates source-centered walks with restart probability 0.15, 400 walks, and walk length 20. Candidate targets are scored by normalized visit frequency. This produces a practical approximation to source-conditioned graph relevance.

3.4 Candidate Sampling and Evaluation

For every source node with hidden positives, the candidate set includes its true hidden targets and up to 100 randomly sampled negative targets that are not edges in the full graph. The random sampler is deterministic under the configured seed.

The ranking is evaluated with Macro Precision@K, HitRate@K, and mean reciprocal rank (MRR). Precision@K measures the average fraction of top- K candidates that are true hidden edges. HitRate@K records whether at least one hidden edge appears in the top- K list. MRR rewards algorithms that place the first positive target early in the ranking.

4 Experiments

The experiment uses the repository defaults: seed 42, test ratio 0.2, $K = 10$, and 100 negative samples per source. The workflow consists of four stages:

Table 1: Full-dataset retrieval performance with $K = 10$ and 100 negative candidates per source.

Algorithm	P@10	HR@10	MRR
Common Neighbors	0.229641	0.711373	0.493400
Jaccard	0.219742	0.698766	0.420111
Adamic-Adar	0.230472	0.714056	0.502139
Personalized PR	0.239297	0.791845	0.551049

1. prepare the train/test edge split from `Wiki-Vote.txt`;
2. evaluate all graph scorers over sampled candidate sets;
3. retrieve the top related nodes for a selected source user;
4. generate summary plots from saved metrics and ranking files.

The implementation is intentionally lightweight: it is a retrieval module for GraphRAG, not a full LLM answering system. This separation makes the retrieval behavior measurable without confounding it with generator quality, prompt design, or answer evaluation.

5 Results

Table 1 reports the saved full-dataset results. Personalized PageRank is the strongest scorer on all three metrics. Adamic-Adar and Common Neighbors are close on Precision@10, but Personalized PageRank improves HitRate@10 substantially, suggesting that random-walk context is better at placing at least one true hidden edge in the top retrieved set.

Figures 1 and 2 visualize the metric-level behavior. The Precision@K curve is useful because a retrieval module may expose different numbers of contexts to a downstream generator; an algorithm that is robust across K is easier to tune in a complete GraphRAG pipeline.

Figures 3 and 4 show two complementary views. The heatmap makes algorithm cutoffs comparable, while the score distribution indicates how well the scorer separates hidden positive edges from random non-edges. This distinction matters for retrieval: even a modest Precision@K can still be useful if positives are consistently shifted toward higher scores.

Figures 5 and 6 connect the quantitative metrics back to graph structure. The degree distribution explains why purely local overlap can be brittle: many nodes have limited visible neighborhoods, while a smaller number of hubs dominate connectivity. The retrieval network example illustrates how the module can provide structural evidence, such as short paths or shared neighbors, for a recommended context node.

6 Conclusion

This project demonstrates a reproducible GraphRAG retrieval module by turning context retrieval into link

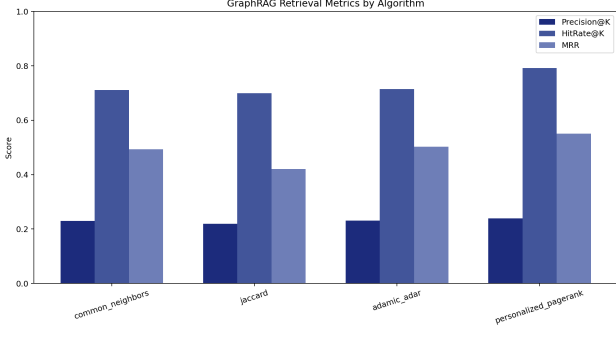


Figure 1: Metric summary across graph retrieval algorithms.

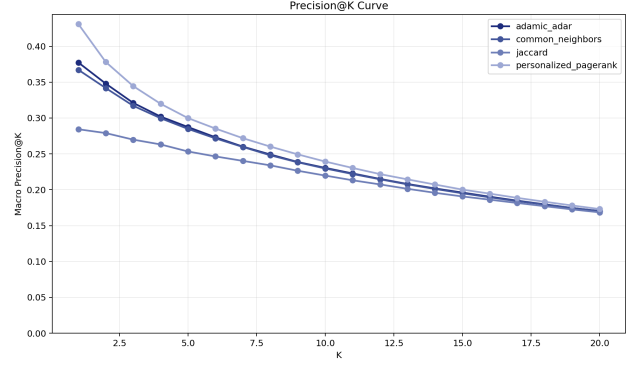


Figure 2: Precision@K curve showing how retrieval quality changes as K increases.

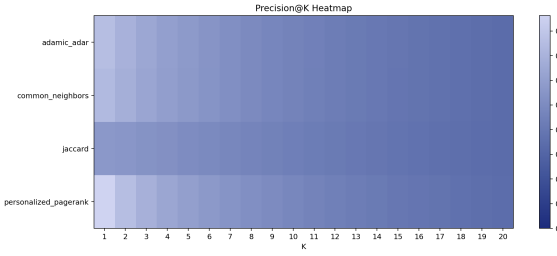


Figure 3: Precision@K heatmap for comparing algorithms and cutoff values.

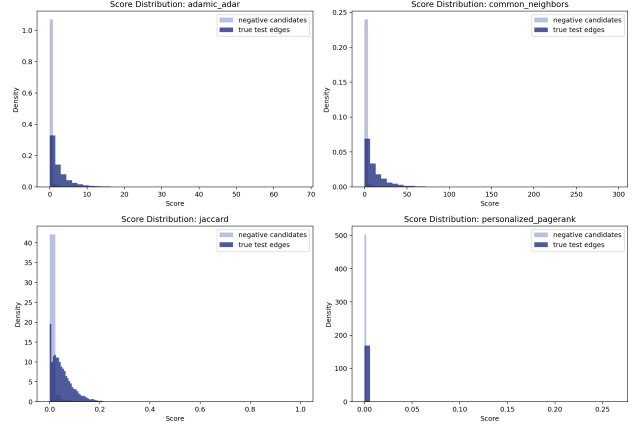


Figure 4: Score distribution for positive hidden edges and sampled negative candidates.

prediction on the Wiki-Vote graph. The results show that approximate Personalized PageRank provides the best retrieval quality among the implemented scorers, outperforming local overlap methods on Precision@10, HitRate@10, and MRR. More broadly, the study supports the value of graph-aware retrieval when relevant context is relational rather than purely textual.

The main limitation is that the system stops at retrieval: it does not yet feed retrieved graph context into an LLM, evaluate generated answers, or compare against dense text retrieval. Future work should add textual node attributes, test graph-plus-vector hybrid retrieval, and measure the downstream effect of retrieved graph context on answer factuality and citation quality.

References

- [1] Lada A. Adamic and Eytan Adar. Friends and neighbors on the web. *Social Networks*, 25(3):211–230, 2003. doi: 10.1016/S0378-8733(03)00009-1. URL [https://doi.org/10.1016/S0378-8733\(03\)00009-1](https://doi.org/10.1016/S0378-8733(03)00009-1).
- [2] Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, Dasha Metropolitansky, Robert Osazuwa Ness, and Jonathan Larson. From local to global: A graph rag approach to query-focused summarization, 2024. URL <https://arxiv.org/abs/2404.16130>.
- [3] Aric A. Hagberg, Daniel A. Schult, and Pieter J. Swart. Exploring network structure, dynamics, and function using networkx. In *Proceedings of the 7th Python in Science Conference*, pages 11–15, 2008. doi: 10.25080/TCWV9851. URL <https://proceedings.scipy.org/articles/TCWV9851>.
- [4] Glen Jeh and Jennifer Widom. Scaling personalized web search. In *Proceedings of the 12th International Conference on World Wide Web*, WWW ’03, pages 271–279, New York, NY, USA, 2003. ACM. doi: 10.1145/775152.775191. URL <https://doi.org/10.1145/775152.775191>.
- [5] Jure Leskovec and Andrej Krevl. SNAP Datasets: Stanford large network dataset collection. <https://snap.stanford.edu/data>, June 2014. URL <https://snap.stanford.edu/data/wiki-Vote.html>.
- [6] Jure Leskovec, Daniel Huttenlocher, and Jon Kleinberg. Predicting positive and negative links in online social networks. In *Proceedings of the 19th International Conference on World Wide Web*, WWW ’10, pages 641–650, New York, NY, USA,

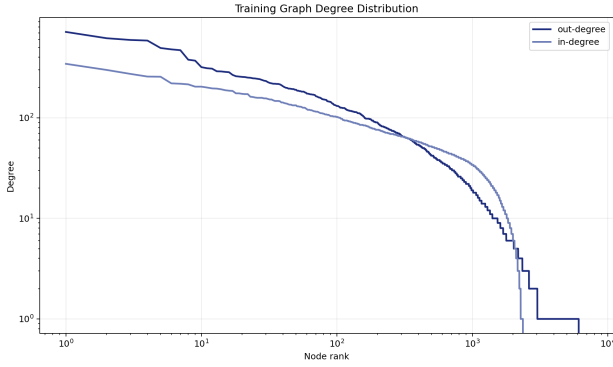


Figure 5: Training graph degree distribution, showing the heavy-tailed structure typical of real social networks.

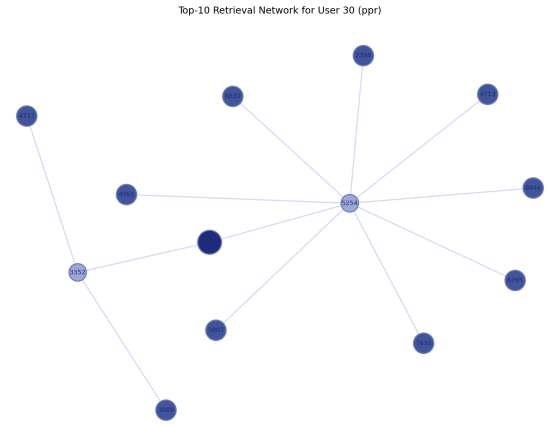


Figure 6: Example Personalized PageRank retrieval network for user 30.

2010. ACM. doi: 10.1145/1772690.1772756. URL <https://doi.org/10.1145/1772690.1772756>.

- [7] Jure Leskovec, Daniel Huttenlocher, and Jon Kleinberg. Signed networks in social media. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, CHI '10, pages 1361–1370, New York, NY, USA, 2010. ACM. doi: 10.1145/1753326.1753532. URL <https://doi.org/10.1145/1753326.1753532>.
- [8] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, Mike Lewis, Wentaoyi Yih, Tim Rocktaschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474, 2020. URL <https://papers.neurips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html>.
- [9] David Liben-Nowell and Jon Kleinberg. The link-prediction problem for social networks. *Journal of the American Society for Information Science and Technology*, 58(7):1019–1031, 2007. doi: 10.1002/asi.20591. URL <https://doi.org/10.1002/asi.20591>.
- [10] Lawrence Page, Sergey Brin, Rajeev Motwani, and Terry Winograd. The pagerank citation ranking: Bringing order to the web. Technical Report SIDL-WP-1999-0120, Stanford InfoLab, 1999. URL <https://ilpubs.stanford.edu/422/>.